CODSOFT Python Programming Internship

Task 3: Password Generator

- Submitted By:

  Gourav Singh Rajpoot

- Internship Domain:

  Python Programming

Project

****Strong Password Generator Using Python****

 # Objective

The main goal of this project is to create a Password Generator using Python. This generator will create random passwords based on what the user wants. The user can choose how long the password is, if it includes characters, how many passwords to generate and check how strong the password is.

 # Problem Statement

Using weak passwords can make your online accounts easy to hack. A strong password generator helps users create passwords with a mix of uppercase letters, lowercase letters, numbers and special characters.

The application should do the following things:

* Create passwords

* Let the user choose the password length

* Support characters

* Create multiple passwords

* Show the password strength

* Handle invalid user input

Tools and Technologies Used

Tool	Purpose
Python 3	Programming Language
VS Code / IDLE	Code Editor
Command Prompt / Terminal	Program Execution
Random Module	Picks characters
String Module	Gives us collections of characters

System Requirements

- **Hardware Requirements**
  * Computer/Laptop
  * Minimum 2 GB RAM

- **Software Requirements**
  * Python 3.x
  * Any Code Editor (VS Code, PyCharm, IDLE)

Algorithm

- **Step 1**

We need to import the modules:

* random

* string

- **Step 2**

We ask the user for the password length.

- **Step 3**

We make sure the password length is least 6 characters.

- **Step 4**

We ask if the user wants to include characters.

- **Step 5**

We ask how passwords the user wants to generate.

- **Step 6**

We generate passwords using:

    * letters

    * Lowercase letters

    * Numbers

    * Special symbols (if the user wants them)

- **Step 7**

We shuffle the characters randomly.

- **Step 8**

We show the generated password.

- **Step 9**

We check the password strength:

* than 8 characters: Weak

* 8 to 11 characters: Medium

* 12 or more characters:

- **Step 10**

We handle invalid inputs using exception handling.

 # Source Code

```
# Strong Password Generator

import random
import string

print("===== Strong Password Generator =====")

try:
  password_length = int(input("Enter password length: "))

  if password_length < 6:
    print("Password length should be at least 6.")

  else:
    include_symbols = input("Include special symbols? (y/n): ").lower()
    total_passwords = int(input("How many passwords do you want to generate? "))

    for i in range(total_passwords):
```

```
password = [
    random.choice(string.ascii_uppercase),
    random.choice(string.ascii_lowercase),
    random.choice(string.digits)
]

if include_symbols == 'y':
    password.append(random.choice(string.punctuation))

all_characters = string.ascii_letters + string.digits

if include_symbols == 'y':
    all_characters += string.punctuation

remaining = password_length - len(password)

password += random.choices(all_characters, k=remaining)

random.shuffle(password)

final_password = "".join(password)

print(f"\nPassword {i+1}: {final_password}")

# Password Strength Checker
if password_length < 8:
    print("Strength: Weak")
```

```
        elif password_length < 12:
            print("Strength: Medium")
        else:
            print("Strength: Strong")

except ValueError:
    print("Please enter valid numeric values.")
```

- **Explanation of the Code**

 # Random Module

We use this module to pick random characters and create unpredictable passwords.

 # String Module

This module gives us predefined sets of characters like:

* Uppercase Letters

* Lowercase Letters

* Digits

* Special Symbols

 # Password Generation

Our program makes sure each password has:

* At one uppercase letter

* At one lowercase letter

* At one digit

* At least one special character (if the user wants it)

 # Password Strength Checker

Our program checks the password strength and says it is:

* Password Length is Less than 8 Characters

So, The Strength of Password is Weak.

* Password Length is 8 to 11 Characters

So, The Strength of Password is Medium.

* Password Length is 12 or More Characters

So, The Strength of Password is Strong.

* Password Length is Less than 8 Characters

So, The Strength of Password is Weak.

* Password Length is 8 to 11 Characters

So, The Strength of Password is Medium.

* Password Length is 12 or More Characters

So, The Strength of Password is Strong.

 # Exception Handling

We use a try-except block to stop the program from crashing when the user enters something invalid.

- **Sample Output**

===== Strong Password Generator =====

Enter password length: 12

Include special symbols? (y/n): y

How passwords do you want to generate? 2

Password 1: A@7mK#9qLp2!

Strength:

Password 2: p$4Xz!8NqR1&

Strength: Strong

 # Features

* We generate passwords

* The user can choose the password length

* We can generate passwords

* The user can choose to include characters

* We check the password strength

* We validate the users input

* We use a command-line interface

 # Advantages

* Our program generates passwords quickly

* It reduces the risk of using passwords

* It is easy to use

* It can generate passwords at once

* It helps people understand how to make their passwords more secure

 # Learning Outcomes

From this project we learned about:

* The basics of Python

* How to generate numbers

* How to work with strings

* How to handle user input

* How to use lists and loops

* How to use statements

* How to handle exceptions

* How to make passwords more secure

* How to work with Python modules

Future Enhancements

* We can add a feature to copy the password to the clipboard

* We can save the passwords to a file

* We can create a graphical user interface

* We can suggest when to change the password

* We can let the user choose custom character sets

* We can integrate it with a password manager

#Conclusion

We successfully created a Strong Password Generator using Python. Our program generates random passwords based on the users preferences checks the password strength and handles invalid inputs. This project shows how we can use Python programming concepts and password security techniques in a real-world application.

Result

We successfully. Tested the Password Generator. Users can generate passwords create multiple passwords, at once include special characters and check the password strength using a simple command-line interface.